“COMMONROOM: A REAL-TIME GUEST CHAT APPLICATION”

Agile Methodology & DevOps Lab (20CP408P)

Agile Lab Project Report

Submitted
By

Maitree Mistry (Roll No: 23BCP181)
Mahek Kamol (Roll No: 23BCP178)

Under the Guidance of
Dr Komal Singh

Department of Computer Engineering,
School of Technology, Pandit Deendayal Energy University,
Gandhinagar 382 426

October 2026

ABSTRACT

This report presents the Agile Lab project Commonroom, a browser-based, guest-first, real-time chat application. A visitor chooses a display name and joins a room without creating an account, reads recent room history, sees who is currently present, switches rooms and exchanges messages live with other connected guests. The application provides three default rooms (`general`, `random` and `help`) and also supports custom room names.

The system is built with React 19, Vite and Tailwind CSS on the front end; Node.js with Express 5 and Socket.IO 4 on the back end; and Redis 7 for bounded message history, presence, rate limiting and cross-instance event delivery. It is deployed as three Docker Compose services behind an Nginx reverse proxy, with the backend and Redis isolated on private networks.

The project was planned and delivered using an Agile, Scrum-style approach: requirements were captured as user stories in a product backlog and delivered in incremental, working, tested releases. Key engineering outcomes include signed 12-hour guest sessions, Zod-validated payloads, Redis-backed rate limiting, idempotent message retries using a client-generated message ID, and a policy of broadcasting a message only after Redis confirms it has been stored. Quality was supported by backend, integration and frontend test suites written with Vitest, Supertest and Testing Library.

In line with the Agile Methodology & DevOps Lab, the project applies DevOps practices end to end: **unit testing, Docker containerisation, an Nginx reverse proxy, and a CI/CD pipeline implemented as a GitHub Actions workflow** that builds and tests every change automatically.

Overall, the project strengthened skills in real-time systems, full-stack development, containerised deployment, automated testing and Agile delivery, while also clarifying the limits of a lightweight chat system, which does not provide verified identity, room privacy or long-term durable storage.

INDEX

CHAPTER 1: INTRODUCTION 1

1.1 About the Project 1
1.2 Prologue / Background 1
1.3 Motivation 1
1.4 Objective 2
1.5 Scope of the Project 2
1.6 Organization of the Report 2

CHAPTER 2: LITERATURE REVIEW 3

2.1 Existing Solutions in the Same Domain 3
2.2 Problem Identification 3

CHAPTER 3: PROPOSED METHODOLOGY AND IMPLEMENTATION 4

3.1 Agile Methodology and Planning 4
3.2 Requirements Analysis 5
3.3 Technology Stack 6
3.4 System Architecture 6
3.5 Guest Identity and Sessions 7
3.6 Joining a Room 7
3.7 Sending and Receiving Messages 8
3.8 Presence Tracking 8
3.9 HTTP and Socket.IO Interfaces 8
3.10 Redis Data Model 9
3.11 Security and Safeguards 9
3.12 Containerised Deployment 10
3.13 Testing Strategy 10
3.14 DevOps Practices and CI/CD Pipeline 11

CHAPTER 4: RESULTS AND DISCUSSION 13

4.1 Delivered Functionality 13
4.2 Test Outcomes 13
4.3 Cache Benchmark 13
4.4 Observed Behaviour 14
4.5 Challenges and Lessons Learned 14

CHAPTER 5: CONCLUSION AND FUTURE SCOPE 15

5.1 Conclusion 15
5.2 Future Scope 15

LIST OF FIGURES

Figure	Title	Section
Figure 3.1	High-level architecture of Commonroom	3.4

NOMENCLATURE

Abbreviation	Full Form
AOF	Append-Only File
AWS	Amazon Web Services
API	Application Programming Interface
CD	Continuous Delivery / Deployment
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
RDB	Redis Database snapshot
SPA	Single-Page Application
TTL	Time To Live
UI	User Interface
UUID	Universally Unique Identifier
US	User Story

CHAPTER 1
INTRODUCTION

1.1 About the Project

Commonroom is a browser-based, guest-first, real-time chat application. A visitor chooses a display name and joins a room without creating an account. They can read recent room history, see who is currently present, switch rooms, and exchange messages live with other connected guests.

The application includes three default rooms (`general`, `random` and `help`) and also supports custom room names. A room is shared by everyone who joins that room name; it is not private or access-controlled.

Commonroom is a lightweight collaboration and demonstration application, not a verified-identity or high-durability messaging platform:

A display name is a label, not proof of identity.

Messages are not end-to-end encrypted.

Redis keeps recent room history but is not a full durable database.

A Redis outage prevents messages from being stored and therefore prevents their broadcast.

1.2 Prologue / Background

This report documents the design, development and testing of Commonroom as the project for the Agile Lab. The work combined full-stack web development with real-time communication, in-memory data storage and container-based deployment, and was organised using Agile practices: a prioritised backlog of user stories, short iterations that each produced a working increment, and continuous testing.

The project brought together several distinct engineering concerns in a single system: browser-side state management, authenticated WebSocket connections, atomic data operations in Redis, input validation and abuse protection, and reproducible deployment with Docker Compose.

1.3 Motivation

Many small teams, classes and events need a quick place to talk. Account-based chat tools add sign-up friction, while ad-hoc solutions often lack basic safeguards such as input validation, rate limiting and duplicate-message protection. At the same time, academic exercises rarely require a system to handle retries, partial failures, multiple browser tabs or hostile input. The motivation for this project was to build a small but realistic real-time application that is quick to join, behaves correctly under those conditions, and is developed the way professional teams work: incrementally, with working software and automated tests at every step.

1.4 Objective

To let a visitor join and chat as a guest with no registration.

To deliver messages and presence updates to all room members in real time.

To keep a bounded recent history that new joiners can read.

To make message sending reliable: retries must not create duplicates, and messages that were not stored must never be broadcast.

To validate all input and protect the service with rate limits and security headers.

To package the system so that it can be started with a single command.

To support the work with automated unit and integration tests.

To practise Agile delivery through a backlog, incremental releases and a clear Definition of Done.

To apply the DevOps practices taught in the course: **GitHub Actions workflows, unit testing, Docker, Nginx and a CI/CD pipeline**.

1.5 Scope of the Project

The scope covers four workstreams: (i) the React front end, including the join screen, room navigation, message list, composer and presence display; (ii) the Express and Socket.IO back end, including guest sessions, room and message events, validation and rate limiting; (iii) Redis-based storage, presence and coordination; and (iv) containerised deployment with Docker Compose and Nginx, together with automated unit and integration tests, a cache benchmark and a CI/CD pipeline built as a GitHub Actions workflow.

The following are explicitly outside the scope: verified identities or accounts, private rooms, user roles and moderation, end-to-end encryption, message search or archiving, and long-term durable storage.

1.6 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 reviews existing chat solutions and the problem areas identified. Chapter 3 describes the Agile methodology, architecture and implementation of each part of the system. Chapter 4 presents the results, test outcomes and lessons learned. Chapter 5 concludes the report and outlines directions for future work, followed by the references cited.

CHAPTER 2
LITERATURE REVIEW

2.1 Existing Solutions in the Same Domain

Real-time chat is a well-established domain. Hosted team-messaging products such as Slack, Microsoft Teams and Discord offer rich features including accounts, private channels, search, file sharing and moderation, but they require sign-up and are designed for sustained, identity-based collaboration. Open protocols such as IRC and Matrix provide decentralised or self-hosted alternatives with their own server and client ecosystems. Backend-as-a-service offerings can supply real-time messaging as a managed feature, at the cost of dependence on an external provider.

At the library level, Socket.IO is a widely used framework for bidirectional, event-based communication over WebSockets with fallbacks, and it includes a Redis adapter that distributes room events between several server instances. Redis itself is an in-memory data store commonly used for caching, counters, sorted sets and publish/subscribe messaging. Studying these building blocks showed that a small chat service can be assembled from standard components without a separate SQL database or message broker.

2.2 Problem Identification

The established products are heavier than needed for a short-lived, low-friction conversation, and simple tutorial-style chat examples tend to omit the safeguards that make a system trustworthy. The following gaps were identified and formed the basis of the project:

Friction: a quick conversation should not require an account.

Trust in identity: the server must not accept a sender ID or name from a message payload.

Reliability: retries after a timeout must not create duplicate messages, and an unstored message must not be shown to other users.

Presence accuracy: a guest with several tabs should not disappear when one tab closes.

Abuse resistance: input must be validated and message and connection rates limited.

Reproducibility: the whole system should start with one command and keep the data store off the public network.

CHAPTER 3
PROPOSED METHODOLOGY AND IMPLEMENTATION

3.1 Agile Methodology and Planning

The project followed a Scrum-style, iterative approach. Work was captured as a product backlog of user stories and delivered in short increments, each ending in a working, tested version of the application. Each increment built on the previous one, so a demonstrable product existed early and grew in capability.

3.1.1 Roles and ceremonies

Element	How it was applied
Product Owner	[Name]: prioritises the backlog and accepts completed stories
Scrum Master	[Name]: facilitates ceremonies and removes blockers
Development team	Maitree Mistry and Mahek Kamol: design, implement and test
Sprint length	[e.g., 1 week]
Sprint planning	Select the highest-priority stories that fit the sprint
Daily stand-up	Brief status: done, next, blockers
Sprint review	Demonstrate the working increment (two browser sessions chatting)
Retrospective	Identify improvements for the next sprint

3.1.2 Product backlog (user stories)

ID	User story	Priority
US-01	As a visitor, I want to enter a display name and join without an account so that I can start chatting quickly.	High
US-02	As a guest, I want to join a room and see recent messages so that I understand the context.	High
US-03	As a guest, I want to send a message that everyone in the room sees instantly.	High
US-04	As a guest, I want to see who is currently in the room.	Medium
US-05	As a guest, I want to switch between rooms or create a custom room.	Medium
US-06	As a guest on a poor connection, I want unconfirmed messages to be retried without duplicates.	Medium
US-07	As an operator, I want input validation and rate limits so that the service resists abuse.	High
US-08	As an operator, I want to run the whole system with one command.	Medium
US-09	As a developer, I want automated tests so that changes can be made safely.	High
US-10	As a developer, I want every push and pull request to be built and	High

ID	User story	Priority
	tested automatically by a CI/CD pipeline (GitHub Actions) so that broken changes are caught early.	

3.1.3 Incremental delivery plan

Increment	Goal	Stories	Outcome
1	Foundation and guest sessions	US-01	Express API, input validation, signed guest sessions, join screen
2	Real-time chat core	US-02, US-03	Socket.IO connection, room join, message send and broadcast, Redis history
3	Presence and rooms	US-04, US-05	Multi-socket presence, room list, room switching
4	Reliability and safeguards	US-06, US-07	Idempotent retries, rate limits, error handling, security headers
5	Packaging, quality and CI/CD	US-08, US-09, US-10	Docker Compose, Nginx, Redis configuration, test suites, benchmark, GitHub Actions workflow

3.1.4 Definition of Done

A story is done when its behaviour works end to end in the running application, input is validated on the server, automated tests cover the new behaviour, the frontend still builds, the change runs correctly under Docker Compose, and the GitHub Actions workflow passes.

3.2 Requirements Analysis

3.2.1 Functional requirements

Create a guest session from a validated display name.

Authenticate each Socket.IO connection with a signed token.

Join, leave and switch rooms; list default and previously used rooms.

Send and receive messages in real time; read recent history on join.

Show current presence, listing each guest once per room.

Retry unconfirmed messages safely.

3.2.2 Non-functional requirements

Security: signed sessions, validated payloads, rate limits, hardened headers, non-root containers, private Redis network.

Reliability: atomic Redis operations, idempotent writes, readiness checks that reflect Redis availability.

Maintainability: separated routes, sockets, repositories, services and validation; automated tests.

Portability: reproducible three-service Docker Compose deployment.

Performance: bounded history (maximum 100 messages per room) and short-lived Redis keys for presence and counters.

3.3 Technology Stack

Area	Technology	Responsibility
Browser UI	React 19, Vite 8, Tailwind CSS 4	Join screen, navigation, message list and composer, presence and connection status
HTTP backend	Node.js (ES modules), Express 5	Guest sessions, room listing, history reads, health and readiness checks
Live communication	Socket.IO 4	Authenticated connections, room membership, message and presence events
Storage and coordination	Redis 7 via ioredis	Message history, room registry, presence, rate counters, adapter pub/sub
Production web server	Nginx	Serves the built frontend and proxies API and WebSocket traffic
Testing	Vitest, Supertest, Testing Library, jsdom	Backend, integration and frontend tests
Orchestration	Docker Compose	Builds and runs frontend, backend and Redis together
CI/CD	GitHub Actions	Runs the build and test workflow automatically on every push and pull request

There is no SQL database or separate message broker. Redis serves as both the shared state store and the Socket.IO adapter transport.

3.4 System Architecture

The browser talks only to Nginx. Nginx serves the static React build and forwards `/api` (HTTP) and `/socket.io` (WebSocket) traffic to the backend. The backend reads and writes Redis for history, presence and rate counters, and uses Redis pub/sub through the Socket.IO adapter so that events can reach clients connected to different backend instances.

```
Browser (React)
      |   HTTP /api and WebSocket /socket.io
      v
Nginx (frontend container)
      |   proxied API and WebSocket traffic
      v
Backend (Express + Socket.IO)
      |   history, presence, rate counters, pub/sub
      v
Redis
```

Figure 3.1: High-level architecture of Commonroom

Only Nginx publishes a host port. The backend is reachable only over Docker's private frontend network, and Redis sits on a separate internal network shared only with the backend.

Path	Purpose
backend/src/app.js	Express middleware and API setup
backend/src/server.js	Configuration, Redis startup, HTTP and Socket.IO lifecycle

Path	Purpose
backend/src/config/env.js	Environment parsing and validation
backend/src/routes/api.js	HTTP endpoints
backend/src/sockets/chat.js	Socket authentication and chat event handlers
backend/src/repositories/	Redis operations for history and presence
backend/src/services/	Guest signing and Redis client setup
backend/src/validation/schemas.js	Zod schemas and normalisation
backend/src/middleware/errors.js	Consistent HTTP error responses
frontend/src/hooks/useChat.js	Socket lifecycle and chat state
frontend/src/components/	Join, layout, messages, users, status, controls
frontend/src/services/	HTTP API adapter and Socket.IO client
docker-compose.yml, redis/redis.conf	Deployment and Redis settings

3.5 Guest Identity and Sessions

1. The browser loads rooms from `GET /api/rooms`.
2. The visitor picks a display name and room; the UI performs basic checks while the backend independently validates.
3. The browser sends `POST /api/guest-session` with the display name.
4. The backend trims and validates the name, generates a random UUID, and issues a 12-hour token containing the guest ID, display name and expiry, signed with HMAC-SHA256 using `GUEST_SESSION_SECRET`.
5. The session and selected room are saved in the tab's `sessionStorage`, so closing the tab clears them.
6. The frontend opens a Socket.IO connection and sends the token in the auth handshake; the server verifies signature and expiry before accepting it.

The server never trusts a user ID or sender name from a message payload. These values come from the verified socket session.

3.6 Joining a Room

The client emits `room:join` and waits for an acknowledgement. The backend applies the room-event rate limit, validates the payload, registers the room, adds the socket to the Socket.IO room, records presence in Redis, reads recent history and current users, then acknowledges with the room data, emits `room:joined` and broadcasts `presence:update`. Joining another room first leaves the previous one.

3.7 Sending and Receiving Messages

1. The browser emits `message:send` with `roomId`, `content` and a UUID `clientMessageId`.
2. The backend confirms the sender joined the room, validates the content and enforces the per-socket rate limit.
3. The backend assigns the server message ID and timestamp, taking sender identity from the session.
4. A single Redis Lua script checks for a retry, records the idempotency key, appends the message, trims history to its maximum and registers the room.
5. Only after Redis confirms the write does the backend broadcast `message:new` and acknowledge the sender.
6. The frontend merges the acknowledgement and broadcast so the message appears once.

If an acknowledgement times out, the composer keeps the content and retries with the same `clientMessageId`. Redis recognises the sender and client ID pair for 24 hours and returns the original message, avoiding a duplicate history entry or broadcast.

3.8 Presence Tracking

Presence is tracked per guest and per socket, so a guest with several tabs stays present until the last socket leaves. Socket presence expires after 90 seconds without a heartbeat; the backend refreshes it every 30 seconds. Leaving a room or disconnecting removes the socket, and stale entries are pruned when users are read.

3.9 HTTP and Socket.IO Interfaces

HTTP responses use a uniform envelope: `{ "success": true, "data": {}, "error": null }`.

Request	Purpose
GET /api/health	Liveness check (does not check Redis)
GET /api/ready	Readiness check; returns 503 if Redis is unavailable
POST /api/guest-session	Validate a name and issue a signed session
GET /api/rooms	Default rooms plus rooms registered in Redis
GET /api/rooms/:roomId/messages?limit=50	Recent messages in chronological order (limit 1 to 100)
Client event	**Purpose**
room:join	Join a room; acknowledgement includes history and users
room:leave	Leave a room
message:send	Send a message; acknowledgement includes the stored message and duplicate status

Server events are `room:joined`, `room:left`, `message:new`, `presence:update` and `system:error`.

3.10 Redis Data Model

Data	Representation	Lifetime
Room history	One Redis list per encoded room name	Latest N messages (default and maximum 100)
Message retry keys	Key per room, sender and client message ID	24 hours
Known rooms	Set chat:rooms	Persistent while stored
Presence	Socket keys, per-user socket sets, room sorted sets, user hashes	90-second socket TTL with heartbeat
Rate limits	Incrementing counters per window	Expire after the window

Room names are Base64URL-encoded in keys so raw user text never forms a key segment. Message append and presence changes use Lua scripts for atomicity. The backend uses three ioredis connections: `cache` for commands, and `pub` and `sub` for the Socket.IO Redis adapter, which distributes events across backend instances but does not store history.

3.11 Security and Safeguards

Signed sessions: HMAC-SHA256 tokens with expiry; tampered, expired or differently signed tokens are rejected.

Validation: Zod schemas trim and normalise input, reject control characters, require UUID client IDs and reject unexpected fields.

Rate limiting: 120 API requests per minute; 20 guest-session creations per minute; Redis-backed socket limits of 30 connection attempts per IP, 12 messages per socket and 20 room events per socket, each per 10 seconds.

HTTP hardening: Helmet headers, a 16 KiB JSON body limit, and Nginx security headers with a Content Security Policy.

Error handling: a consistent JSON envelope that hides internal error details; Pino logs redact tokens, passwords, cookies and authorisation data.

Infrastructure: Redis requires a password, disables `FLUSHDB`, `FLUSHALL`, `CONFIG`, `SHUTDOWN` and `DEBUG`, has no published port and sits on an internal network. The backend runs as the unprivileged `node` user with `no-new-privileges`.

These measures protect the application; they do not replace an edge proxy, production abuse monitoring or deployment-specific controls.

3.12 Containerised Deployment

Docker Compose defines three services:

frontend: builds the React app, serves it from Nginx, publishes host port 3000, proxies `/api` and `/socket.io/` (with WebSocket upgrade headers), and exposes `/healthz`.

backend: multi-stage Node 24 Alpine image with production-only dependencies, non-root user, readiness health check on `/api/ready`, not published to the host.

redis: pinned `redis:7.4.2-alpine`, password authentication, checked-in configuration with AOF (fsync every second) and RDB snapshots, data on the named volume `realtime-chat-redis-data`.

Two networks isolate the services: `frontend_net` for frontend and backend, and `redis_net` (`internal: true`) for backend and Redis. Startup is ordered by health conditions: Redis, then backend, then frontend.

```
Copy-Item .env.example .env
# set REDIS_PASSWORD and GUEST_SESSION_SECRET (32+ characters)
docker compose up --build -d
docker compose ps
```

The application is then opened at `http://localhost:3000` in two browser sessions using different display names.

3.13 Testing Strategy

Unit testing is a core part of the project. Testing runs at three levels: unit tests for validation and signing, API tests with fake dependencies, and integration tests against real Redis and Socket.IO. Frontend behaviour is tested separately in jsdom with Testing Library.

File	What it verifies
validation.test.js	Name validation, room normalisation, message trimming and line endings, length limits, UUID IDs, strict payloads
guest-sessions.test.js	Valid session creation and verification; rejection of modified, expired or differently signed tokens
api.test.js	Liveness and readiness, room and history responses, invalid inputs, session creation, safe JSON errors
redis.integration.test.js	Room key isolation, idempotent writes, chronological bounded retention, multi-socket presence
sockets.integration.test.js	Room isolation, invalid payloads, history on join, retry de-duplication, no broadcast on storage failure, presence across sockets
JoinScreen.test.jsx	Name validation, room normalisation, room choices
MessageComposer.test.jsx	Enter and Shift+Enter behaviour; retry with the same message ID
chatApi.test.js	API response handling; mapping the token to the socket auth field
useChat.test.jsx	Socket event contract, acknowledgement and broadcast de-duplication, room switching, room-filtered presence

The Redis repository and Socket.IO integration tests use an authenticated test Redis at `127.0.0.1:6389` by default. If it is unavailable they are explicitly skipped, so a green run without Redis does not validate those paths. A disposable instance can be started with:

```
docker run --rm -d --name commonroom-test-redis -p 6389:6379 redis:7.4.2-
alpine redis-server --requirepass backend-test-password
```

The suites are run with:

```
cd backend && npm test
cd frontend && npm test && npm run build
```

3.14 DevOps Practices and CI/CD Pipeline

Because this project was carried out for the Agile Methodology & DevOps Lab, the delivery process was treated as part of the deliverable, not only the application. The following practices from the course were applied to Commonroom.

Practice	Tool	How it was applied
Continuous Integration	GitHub Actions	A workflow stored in the repository's .github/workflows/ directory runs automatically when code is pushed or a pull request is opened
Unit and integration testing	Vitest, Supertest, Testing Library	The workflow runs the backend and frontend test suites, so a change that breaks behaviour is caught before it is merged
Containerisation	Docker, Docker Compose	Frontend, backend and Redis each have a container definition, so the same images run on any machine with one command
Reverse proxy	Nginx	Serves the built frontend and proxies API and WebSocket traffic; it is the only service that publishes a host port
Continuous Delivery readiness	Docker Compose	A change that passes the pipeline is in a verified, buildable state that can be started and demonstrated immediately

3.14.1 GitHub Actions workflow

The CI/CD pipeline is defined as a GitHub Actions workflow and runs on every push and pull request. Its stages are:

1. **Trigger:** a push or pull request starts the workflow on a clean GitHub-hosted runner.
2. **Install:** backend and frontend dependencies are installed from their lock files.
3. **Test:** the unit and integration test suites are run with `npm test`.
4. **Build:** the frontend production bundle is built with `npm run build`, and the Docker images are built from the Dockerfiles.
5. **Report:** the run is marked passed or failed on the commit or pull request, giving the team immediate feedback.

3.14.2 Link to Agile practice

The pipeline puts two Agile ideas into practice. First, it gives the short feedback loop that incremental delivery depends on: every small change is verified within minutes. Second, it enforces the Definition of Done automatically, since a story is not finished until the workflow

passes. Together with Docker and Nginx, it means the application can be built, tested and run in the same way on every developer’s machine and in the pipeline.

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Delivered Functionality

By the end of the final increment, every story in the product backlog had been implemented and covered by tests, giving a complete, working application that starts with a single Docker Compose command.

Story	Result
US-01 Guest join	Delivered: validated display name, signed 12-hour session
US-02 History on join	Delivered: up to 100 recent messages returned on join
US-03 Real-time messaging	Delivered: stored in Redis first, then broadcast and acknowledged
US-04 Presence	Delivered: unique guests per room, multi-tab safe, heartbeat expiry
US-05 Rooms	Delivered: three default rooms plus custom rooms, with leave-then-join switching
US-06 Safe retries	Delivered: idempotent writes using clientMessageId for 24 hours
US-07 Safeguards	Delivered: validation, HTTP and socket rate limits, hardened headers
US-08 One-command run	Delivered: three-service Docker Compose deployment
US-09 Automated tests	Delivered: backend and frontend suites as listed in Section 3.13
US-10 CI/CD pipeline	Delivered: GitHub Actions workflow that builds and tests every push and pull request (Section 3.14)

4.2 Test Outcomes

Record the results of your own run here before submitting.

Suite	Command	Result
Backend (with test Redis running)	npm test	[passed / total]
Frontend	npm test	[passed / total]
Frontend build	npm run build	[success / failure]
Docker deployment	docker compose ps	[all services healthy]
GitHub Actions workflow	Latest workflow run	[passed / failed]

4.3 Cache Benchmark

`backend/src/scripts/cache-benchmark.js` compares the repository's Redis history read (`LRANGE` plus JSON parsing) with a synthetic in-memory JSON clone. It seeds up to 100 deterministic messages in a temporary room, verifies both paths return identical arrays, warms up, reports total, average, median and p95 latency, and removes its keys.

```
docker compose exec backend node src/scripts/cache-benchmark.js
docker compose exec -e BENCHMARK_OPERATIONS=2000 backend node
src/scripts/cache-benchmark.js
```

Measured output: [paste your results here].

This demonstrates the Redis access path only. It is not a Redis-versus-database comparison: the Redis path includes a container network round-trip while the baseline is in-process, and results vary with machine and load.

4.4 Observed Behaviour

With two browser sessions using different display names in the same room, messages appear in both sessions as soon as Redis confirms the write. If Redis is unavailable, `/api/ready` returns 503 and messages are not stored or broadcast, which is the intended behaviour.

4.5 Challenges and Lessons Learned

Duplicate messages: an acknowledgement and a broadcast can both deliver the same message. Merging by message ID on the client and idempotent keys on the server solved this.

Multi-tab presence: tracking by socket as well as by guest prevents a guest vanishing when one tab closes.

Atomicity: combining the retry check, append, trim and room registration in one Lua script avoids partial writes.

Honest failure: broadcasting only after a successful write keeps users from seeing messages that were never stored.

CI/CD lesson: running the tests and builds automatically in GitHub Actions caught mistakes that were easy to miss locally, and made the Definition of Done objective instead of a matter of memory.

Agile lesson: small increments that could be demonstrated with two browsers surfaced these problems early, when they were cheap to fix, and the Definition of Done kept testing from being postponed.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

Commonroom meets its goal of a lightweight, guest-first real-time chat that is quick to join, protected against common input and abuse problems, and simple to deploy. Redis provides bounded history, presence, rate limiting and cross-instance delivery, while Socket.IO supplies live communication. Delivering the product in small, tested increments, with working software at the end of each, kept the design simple and the quality visible. The DevOps practices from the course (GitHub Actions workflows, unit testing, Docker, Nginx and a CI/CD pipeline) made each increment repeatable to build, test and run.

The project also clarified its own boundaries. Display names and signed sessions give continuity, not verified identity; there is no room privacy, moderation or end-to-end encryption; history is bounded and there is no archive or search; and Redis is required for startup and runtime, so there is no delivery guarantee through an outage. AOF, snapshots and a Docker volume are local persistence, not backup or disaster recovery. The application is therefore suitable as a demonstration and collaboration tool, with clearly documented steps for hardening it before public production use.

5.2 Future Scope

Cloud deployment (AWS or Azure): take the containerised application to the cloud, for example by running the Docker images on AWS (ECS or EC2) or Azure (Container Apps or App Service for Containers), placing a managed load balancer in front of Nginx, and replacing the self-hosted Redis with a managed service such as Amazon ElastiCache or Azure Cache for Redis.

Continuous deployment to the cloud: extend the GitHub Actions workflow so that a passing build on the main branch pushes images to a container registry and deploys them automatically, completing the CI/CD pipeline.

TLS termination, managed secrets and external abuse controls for public deployment.

Monitoring, alerting, backups and a retention policy.

A durable database if long-term history and search are needed.

Private rooms, optional accounts and moderation tools.

Horizontal scaling tests of multiple backend instances using the existing Redis adapter.

REFERENCES

[1] Socket.IO, “Socket.IO Documentation,” 2026.

[2] Redis Ltd., “Redis Documentation,” 2026.

[3] OpenJS Foundation, “Express Documentation,” 2026.

[4] Meta Platforms, Inc., “React Documentation,” 2026.

[5] Docker, Inc., “Docker Compose Documentation,” 2026.

[6] Vitest, “Vitest Documentation,” 2026.

[7] K. Schwaber and J. Sutherland, “The Scrum Guide,” 2020.

[8] K. Beck et al., “Manifesto for Agile Software Development,” Agile Alliance, 2001.